

P1147R0

Printing volatile Pointers

Published Proposal, 2018-10-07

This version:

<https://wg21.link/P1147R0>

Issue Tracking:

[GitHub](#)

Author:

[Bryce Adelstein Lelbach](#) (NVIDIA)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/wg21_p1147_printing_volatile_pointers/blob/master/printing_volatile_pointers.bs

Table of Contents

1 Introduction

2 Wording

3 Acknowledgements

§ 1. Introduction

Printing pointers to `volatile` types with standard library output streams has unexpected results. Consider the following code:

```

#include <iostream>

int main()
{
    int* p0 = reinterpret_cast<int*>(0xdeadbeef);
    volatile int* p1 = reinterpret_cast<volatile int*>(0xdeadbeef);

    std::cout << p0 << std::endl;
    std::cout << p1 << std::endl;
}

```

This produces the following output:

```

0xdeadbeef
1

```

What happened here? Well, `basic_ostream` has an `operator<<` for both `const void*` and `bool`:

27.7.5.1 Class template `basic_ostream`

[ostream]

```

namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_ostream : virtual public basic_ios<charT, traits> {
    public:
        // ...

        // [ostream.formatted], formatted output
        // ...
        basic_ostream<charT, traits>& operator<<(bool n);
        basic_ostream<charT, traits>& operator<<(const void* p);
        // ...

        // ...
    };
}

```

For `std::cout << p0`, `p0` (an `int*`) is implicitly converted to `const void*` and `operator<<(const void*)` is called. However, for `std::cout << p1`, the `operator<<(const void*)` overload is not a match, as it discards qualifiers. Instead, the best match is `operator<<(bool)`, so `p1` (a `volatile int*`) is implicitly converted to `bool` and `operator<<(bool)` is called.

I suggest we add a new `operator<<(const volatile void*)` overload that `const_casts` away `volatile` and calls the `operator<<(const void*)` overload. Initially, I explored modifying the

underlying <locale> methods and changing the existing `operator<<(const void*)` to `operator<<(const volatile void*)`, however, early feedback from other committee members pushed me away from this direction, as there was concern that modifying <locale> would be an ABI breaking change.

Note that `const_casting` away `volatile` is safe here. We are not accessing or printing the value of the object that the pointer points to, we are just printing the value of the pointer itself.

§ 2. Wording

NOTE: The  character is used to denote a placeholder section number which the editor shall determine.

Modify [\[ostream.inserters.arithmetic\]](#) as follows:

27.7.5.2.2 Arithmetic inserters

[ostream.inserters.arithmetic]

```
operator<<(bool val);
operator<<(short val);
operator<<(unsigned short val);
operator<<(int val);
operator<<(unsigned int val);
operator<<(long val);
operator<<(unsigned long val);
operator<<(long long val);
operator<<(unsigned long long val);
operator<<(float val);
operator<<(double val);
operator<<(long double val);
operator<<(const void* val);
```

- 1 *Effects:* The classes `num_get<>` and `num_put<>` handle locale-dependent numeric formatting and parsing. These inserter functions use the imbued `locale` value to perform numeric formatting. When `val` is of type `bool`, `long`, `unsigned long`, `long long`, `unsigned long long`, `double`, `long double`, or `const void*`, the formatting conversion occurs as if it performed the following code fragment:

```
bool failed = use_facet<
    num_put<charT, ostreambuf_iterator<charT, traits>>
    >(getloc()).put(*this, *this, fill(), val).failed();
```

```
basic_ostream<charT, traits>& operator<<(const volatile void* val);
```

- 2 *Effects:* Equivalent to return `operator<<(const_cast<const void*>(val))`;

§ 3. Acknowledgements

Thanks to JF Bastien, Marshall Clow, Billy O’Neal, and Louis Dionne for providing early review and feedback of this paper.